UD03
ESTRUCTURAS DE CONTROL

Francisco Lázaro Pérez Vidal

fl.perezvidal@edu.gva.es

0.INDICE

1. Estructura secuencial
2. Estructuras de selección
3. Estructuras de repetición
4. Estructuras de salto

1.ESTRUCTURA SECUENCIAL

Definición

Java es un lenguaje de programación estructurado, lo que significa que posee distintas estructuras que permiten organizar el código:

- Estructura secuencial
- Estructuras de selección (condicionales)
- Estructuras de repetición (bucles)

Comencemos por la estructura secuencial, que es la que nos indica que, en un programa, las sentencias se ejecutan una después de otra, en el orden en que aparecen escritas dentro del programa.

Las instrucciones se suelen agrupar en bloques, cuyo inicio se marca con la llave de apertura ({) y cuyo fin se marca con el carácter de llave de cierre (}).

1.ESTRUCTURA SECUENCIAL

Ejemplo: Programa que pide 2 números enteros y los muestra por pantalla

```
public class Secuencial {

	public static void main(String[] args){

		//Declaración de variables
		int n1, n2;
		Scanner sc = new Scanner(System.in);

		//Leer el primer número
		System.out.println("Introduce un número entero: ");
		n1 = sc.nextInt();

		//Leer el segundo número
		System.out.println("Introduce otro número entero: ");
		n2 = sc.nextInt();

		//Mostrar resultado
		System.out.println("Los números son: " + n1 + " y " + n2);
	}
}
```

2.ESTRUCTURAS DE SELECCIÓN

Definición

La estructura de selección o condicional permite al programa bifurcar el flujo de ejecución de las instrucciones dependiendo del valor de una expresión (condición).

La condición debe ser una expresión booleana, es decir, debe dar cómo resultado un valor booleano (`true` o `false`)

Se puede diferenciar entre 3 tipos de estructuras de selección:

Estructuras de selección	Código
Selección simple	`if(condición)…`
Selección doble	`if(condición)… else…`

Selección múltiple	if… else… encadenados
	`(if…else…if…else)`
	o
	`switch case`

2.ESTRUCTURAS DE SELECCIÓN

Ejemplo selección simple: determinar si un usuario es mayor de edad

```
public class Edades {

	public static void main(String[] args) {

		Scanner sc = new Scanner(System.in);
		int edad=0;

		System.out.println("Introduzca la edad: ");
		edad = sc.nextInt();

		//Comprobar si es mayor de edad
		if (edad >= 18){
			System.out.println("Eres mayor de edad.");
		}
	}

}
```

Si el bloque de instrucciones tiene una sola instrucción no es necesario escribir las llaves { } aunque para evitar confusiones se recomienda escribir las llaves siempre.

2.ESTRUCTURAS DE SELECCIÓN

Selección doble y selección múltiple

Se evalúa la condición y, si se cumple, se ejecuta un bloque de instrucciones. Si no se cumple, se ejecuta otro bloque de instrucciones. La selección múltiple consiste en encadenar sentencias dobles.

Selección doble

```
if (expresión booleana) {
        instrucciones 1;
}
else{
        instrucciones 2;
}
```

Selección múltiple

```
if (expresion_booleana1){
        instrucciones 1;
}
else if (expresion_booleana2){
        instrucciones 2;
}
else if (expresion_booleana3){
        instrucciones 3;
}
else{
        instrucciones 4;
}
```

2.ESTRUCTURAS DE SELECCIÓN

Ejemplo selección múltiple: saludar en función de la hora

```
public class Saludo {

	public static void main(String[] args) {

		Scanner sc = new Scanner(System.in);
		int hora=0;

		System.out.println("Introduzca una hora (un valor entero): ");
		hora = sc.nextInt();

		if (hora >= 0 && hora < 12)
			System.out.println("Buenos días");
		else if (hora >= 12 && hora < 21)
			System.out.println("Buenas tardes");
		else if (hora >= 21 && hora < 24)
			System.out.println("Buenas noches");
		else
			System.out.println("Hora no válida");
	}

}
```

2.ESTRUCTURAS DE SELECCIÓN

Selección múltiple: instrucción switch

Además de anidar sentencias dobles, se puede conseguir una selección múltiple utilizando la instrucción switch, que tiene el siguiente prototipo:

Selección múltiple con switch

```
switch (expresión){
    case valor 1:
        instrucciones;
        break;
    case valor 2:
        instrucciones;
        break;
    . . .
    default:
        instrucciones;
}
```

En esta expresión, es frecuente preguntar por una variable de tipo int, char o incluso String

2.ESTRUCTURAS DE SELECCIÓN

Ejemplo selección múltiple con switch: identificar número del mes

```
public static void main(String[] args) {

            int mes;
            Scanner sc = new Scanner(System.in);

            System.out.print("Introduzca un numero de mes: ");
            mes = sc.nextInt();

            switch (mes)
            {
               case 1:   System.out.println("ENERO");      break;
               case 2:   System.out.println("FEBRERO");    break;
               case 3:   System.out.println("MARZO");      break;
               case 4:   System.out.println("ABRIL");      break;
               case 5:   System.out.println("MAYO");       break;
               case 6:   System.out.println("JUNIO");      break;
               case 7:   System.out.println("JULIO");      break;
               case 8:   System.out.println("AGOSTO");     break;
               case 9:   System.out.println("SEPTIEMBRE"); break;
               case 10:  System.out.println("OCTUBRE");    break;
               case 11:  System.out.println("NOVIEMBRE");  break;
               case 12:  System.out.println("DICIEMBRE");  break;
               default : System.out.println("Mes no válido");
            }
}
```

2.ESTRUCTURAS DE SELECCIÓN

Comparativa switch VS if.. else.. encadenados: identificar número del mes

```java
public class Meses {

	public static void main(String[] args) {

		int mes;
		Scanner sc = new Scanner(System.in);

		System.out.print("Introduzca un numero de mes: ");
		mes = sc.nextInt();

		switch (mes)
		{
			case 1:  System.out.println("ENERO");      break;
			case 2:  System.out.println("FEBRERO");    break;
			case 3:  System.out.println("MARZO");      break;
			case 4:  System.out.println("ABRIL");      break;
			case 5:  System.out.println("MAYO");       break;
			case 6:  System.out.println("JUNIO");      break;
			case 7:  System.out.println("JULIO");      break;
			case 8:  System.out.println("AGOSTO");     break;
			case 9:  System.out.println("SEPTIEMBRE"); break;
			case 10: System.out.println("OCTUBRE");    break;
			case 11: System.out.println("NOVIEMBRE");  break;
			case 12: System.out.println("DICIEMBRE");  break;
			default : System.out.println("Mes no válido");
		}
	}
}
```

```java
if (mes == 1){
		System.out.println("ENERO");
}else if (mes == 2){
		System.out.println("FEBRERO");
}else if (mes == 3){
		System.out.println("MARZO");
}else if (mes == 4){
		System.out.println("ABRIL");
}else if (mes == 5){
		System.out.println("MAYO");
}else if (mes == 6){
		System.out.println("JUNIO");
}else if (mes == 7){
		System.out.println("JULIO");
}else if (mes == 8){
		System.out.println("AGOSTO");
}else if (mes == 9){
		System.out.println("SEPTIEMBRE");
}else if (mes == 10){
		System.out.println("OCTUBRE");
}else if (mes == 11){
		System.out.println("NOVIEMBRE");
}else if (mes == 12){
		System.out.println("DICIEMBRE");
}else{
		System.out.println("Mes no válido");
}
```

3.ESTRUCTURAS DE REPETICIÓN

Definición

Las estructuras de repetición permiten ejecutar de forma repetida un bloque específico de instrucciones.

Bucle while (MIENTRAS)

Las instrucciones se repiten mientras la condición sea CIERTA.

La condición se comprueba al principio del bucle, por lo que las acciones se pueden ejecutar 0 o más veces.

En cada iteración se evalúa la condición y, si el resultado es FALSO, las instrucciones del bucle no se ejecutan y el programa sigue ejecutándose por la siguiente instrucción después del bucle.

3.ESTRUCTURAS DE REPETICIÓN

Bucle while (MIENTRAS). Ejemplo

```
public static void main(String[] args) {

    Scanner s = new Scanner(System.in);

    int dato = 0;

    //Leemos datos hasta que uno sea negativo...
    while (dato >= 0) {
        System.out.println("Introduzca dato:");
        dato = s.nextInt();
    }

    System.out.println("FIN!");

}
```

3.ESTRUCTURAS DE REPETICIÓN

Bucle do while (REPITE…MIENTRAS)

Las instrucciones se repiten mientras la condición sea CIERTA.

La condición se comprueba al final del bucle, por lo que las acciones se ejecutarán al menos 1 vez.

La ejecución del bucle sigue los siguientes pasos:

1. Se ejecutan las instrucciones a partir del `do{`
2. Se evalúa la condición
3. Si es FALSA, el programa sigue ejecutándose por la siguiente instrucción después del bucle.
4. Si el resultado es CIERTO, se vuelve al paso 1.

3.ESTRUCTURAS DE REPETICIÓN

Bucle do while (REPITE…MIENTRAS). Ejemplo

```
public double perimetroCirculo() {

        Scanner teclado = new Scanner(System.in);

        double radio;

        do {
                System.out.print("Teclee un valor del radio > 0: ");
                radio = teclado.nextDouble();

        } while (radio <= 0);

        return 2 * Math.PI * radio;
}
```

3.ESTRUCTURAS DE REPETICIÓN

Bucle FOR

Hace que un bloque de instrucciones se repitan un número determinado de veces mientras se cumpla una condición.

La estructura general de una instrucción for en Java es la siguiente:

```
for( inicialización; condición; incremento/decremento){
                instrucción 1;
                ..........
                instrucción N;
}
```

A continuación de la palabra "`for`", debe haber siempre 3 zonas:

- *Inicialización:* damos valor inicial a una variable que controlará el bucle.
- *Condición:* condición de permanencia en el bucle.
- *Incremento o decremento:* modificación de la variable que controla el bucle.

3.ESTRUCTURAS DE REPETICIÓN

Bucle FOR. Ejemplos

```java
//Contamos 0 1 2 3 4
for (int i=0; i < 5; i++) {
    System.out.print(i+" ");
}

//Contamos 5 4 3 2 1
for (int i=5; i > 0; i--) {
    System.out.print(i+" ");
}

//Escribimos "Holi wankenobi" 4 veces
for (int i=1; i <= 10; i*=2) {
    System.out.println("Holi wankenobi");
}

//Numeros pares del 0 al 100
for (int i=0; i <= 100; i++) {
    if (i % 2 == 0) {
        System.out.println("Numero par: "+i);
    }
}
```

3.ESTRUCTURAS DE REPETICIÓN

Bucles infinitos

Java permite construir bucles infinitos, los cuales se ejecutarán indefinidamente, a no ser que provoquemos su interrupción. Por ejemplo:

```
for(;;){
        instrucciones
}
```

```
for(;true;){
        instrucciones
}
```

```
while(true){
        instrucciones
}
```



3.ESTRUCTURAS DE REPETICIÓN

Bucles anidados

Bucles anidados son aquellos que incluyen instrucciones for, while o do-while, unas dentro de otras, de forma que en 1 iteración del bucle externo se ejecuta completamente el bucle interno.

Para el caso de los bucles FOR, hay que tener en cuenta que las variables de control que utilicemos han de ser distintas. Por ejemplo:

```
for (int i=0; i < 3; i++) {
    for (int j=0; j < 2; j++) {
        System.out.println("i="+i+"j="+j);
    }
}
```

```
i=0j=0
i=0j=1
i=1j=0
i=1j=1
i=2j=0
i=2j=1
```

4.ESTRUCTURAS DE SALTO

Sentencia break

Se utiliza para interrumpir la ejecución y salir de un bucle o de un switch. Por ejemplo:

```
for (int i = 0; i < 10; i++) {
    System.out.println("Dentro del bucle");
    break;
    System.out.println("Nunca lo escribira");
}
System.out.println(“Tras el bucle”);
```

Cuando el programa alcance la sentencia *break*, detendrá la ejecución del bucle y saldrá de él. Por lo tanto, el resultado por pantalla será:

Dentro del bucle

Tras el bucle

4.ESTRUCTURAS DE SALTO

Sentencia continue

La sentencia *continue* solo puede aparecer dentro de un bucle y se utiliza para saltar a una nueva iteración del mismo, ignorando el resto de instrucciones que pudiera contener.

```
for (int i = 0; i < 10; i++) {
    System.out.println("Dentro del bucle");
    continue;
    System.out.println("Nunca lo escribira");
}
```

Cada vez que el programa alcance la sentencia *continue*, saltará a una nueva iteración del bucle y por tanto la última sentencia *System.out.println* nunca llegará a escribirse.

El resultado del programa será escribir 10 veces *“Dentro del bucle”.*